Open Source Architecture Appendix

Chen Xingqiang · github.com/chenxingqiang · module-level architecture with one project overview per page

Dashed frame = architecture zone
Solid frame = subsystem
Box = module and responsibility

CONTENTS · one architecture per page

- **TraceSmith** — GPU profiling & replay
- **YiRage** — RuntimeFusion
- **JAX-sklearn** — transparent JAX acceleration
- **Secret-Learn** — FL · SS · SL
- **YouTube-DNN** — candidate generation
- **Research CLI** — agent workflow
- **Questions-Gen** — three-stage training

READING GUIDE

Dashed frames are architecture zones, solid inner frames are subsystems, and boxes are concrete modules.

Each page states which parts the system contains and where their responsibility boundaries lie.

TraceSmith — GPU Profiling & Replay Architecture

Applications, pybind11 binding, C++17 core, GPU backends, and output formats

Dashed frame = architecture zone
Solid frame = subsystem
Box = module and responsibility

PROJECT OVERVIEW

TraceSmith is a cross-platform GPU profiling and deterministic replay system for AI compiler, framework, and driver engineers.
Its architecture separates applications, the pybind11 binding, a C++17 core, GPU backends, and output formats.

APPLICATION · user entry

- **C++ Application** — in-process capture
- **Python Application** — in-process capture
- **C++ CLI** — tracesmith; profile · record · view
- **Python CLI** — tracesmith-cli; export · analyze · replay
- **External UI** — Perfetto UI; Chrome tracing

BINDINGS · pybind11 boundary

- **_tracesmith.so** — python/src/bindings.cpp; pybind11 native extension

CORE LIBRARY · C++17

Cluster Module

- **GPUTopology** — NVLink · PCIe · NVML
- **MultiGPUProfiler** — cross-device capture
- **TimeSync** — NTP / PTP / CUDA
- **NCCLTracker** — collective ops
- **ClockCorrelator** — clock alignment
- **CommAnalysis** — communication matrix

Capture Module

- **CUPTIProfiler** — NVIDIA CUDA
- **MCPTIProfiler** — MetaX MACA
- **MetalProfiler** — Apple Metal
- **MemoryProfiler** — GPU memory
- **AscendProfiler** — Huawei CANN
- **createProfiler()** — nsys · mcTracer; xctrace · msprof

State Module

- **GPUStateMachine** — state analysis
- **InstructionStream** — dependency graph
- **TimelineBuilder** — ordered events
- **TimelineViewer** — ASCII rendering
- **PerfettoExporter** — JSON
- **PerfettoProtoExporter** — Protobuf

Replay Module

- **ReplayEngine** — core orchestrator
- **StreamScheduler** — operation order
- **OperationExecutor** — launch replay
- **DeterminismChecker** — result compare
- **FrameCapture** — RenderDoc-style

Common Module

- **Types** — TraceEvent · EventType; KernelParams
- **RingBuffer** — lock-free SPSC
- **StackCapture** — libunwind
- **XRayImporter** — LLVM XRay
- **Timestamps** — ns precision
- **TraceMetadata** — DeviceInfo

Format Module

- **SBTWriter** — binary writer
- **SBTReader** — binary reader
- **.sbt** — header + events; counters

Configuration

- **ProfilerConfig** — buffer · overflow
- **ReplayConfig** — mode · speed
- **StackCaptureConfig** — stack policy
- **NCCLTrackerConfig** — collective policy

GDB Module

- **RSPHandler** — GDB protocol
- **ProcessController** — ptrace / Mach
- **GPUDebugEngine** — breakpoints · replay
- **tracesmith-gdbserver** — v0.10.0

GPU BACKENDS · system interfaces

- **NVIDIA CUDA** — CUPTI SDK
- **Apple Metal** — Metal API
- **MetaX MACA** — MCPTI SDK
- **NCCL** — distributed
- **Nsight Systems** — nsys
- **mcTracer** — MetaX
- **xctrace** — Apple
- **libunwind** — host stacks
- **Perfetto SDK** — trace export

OUTPUTS · trace artifacts

- **trace.sbt** — binary format
- **trace.json** — Perfetto JSON
- **trace.perfetto** — Protobuf
- **graph.dot** — Graphviz
- **ASCII Timeline** — terminal
- **Comm Matrix** — heatmap JSON

The C++17 core contains capture, state, format, replay, cluster, configuration, and GDB. Applications, bindings, backends, and outputs are its boundaries.

YiRage (YPK) — RuntimeFusion Architecture

Inference engine, RuntimeFusion, search plan, backend manager, implementations, and hardware

Dashed frame = architecture zone
Solid frame = subsystem
Box = module and responsibility

PROJECT OVERVIEW

YiRage is a multi-backend LLM kernel compiler and runtime centered on graph superoptimization and RuntimeFusion.

The architecture separates the inference engine, RuntimeFusion, search plans, the backend manager, backend implementations, and hardware targets.

INFERENCE ENGINE · scheduling owner

- **vLLM** — continuous batch · PagedAttention
- **SGLang** — RadixAttention · extend meta
- **Decode Metadata** — block_table · seq_lens · radix hit · SM budget · batch

RuntimeFusion · RF.step(inputs, meta)

- **RF.step** — select 0..N capsules per decode step
- **FusionCapsule** — MLP gate_up + down · yirage.core
- **FusionCapsule** — segment pipeline · gate_up + down
- **Native Fallback** — engine native operators

SEARCH & PLAN · offline / warm-up

- **superoptimize** — KN / TB graph search
- **FusionPlan** — µGraph · profile + fingerprint
- **MuGraph cache** — shape bucket restore
- **Ray Coordinator** — optional distributed search
- **Serving Tiers** — seed_verify · full_tb · full_tb_ray

PYTHON API & BACKEND MANAGER

- **KNGraph / TBGraph** — kernel transpiler
- **BackendRegistry** — search + execute · same backend
- **yirage.core** — Cython runtime
- **yirage.serving** — RF · Capsule · cert
- **MACA Config** — warp = 64 · mxcc
- **CPU MLIR JIT** — optional

BACKEND IMPLEMENTATIONS

- **CUDA** — nvcc · warp 32
- **MACA** — mxcc · warp 64
- **CPU** — OpenMP · MKL
- **MPS** — Apple GPU
- **Ascend** — CANN
- **ROCm** — AMD GPU
- **Triton** — DSL kernels
- **+ extra** — TPU · XPU · FPGA · MLIR

HARDWARE TARGETS

- **NVIDIA GPU**
- **MetaX C500/C550**
- **x86 / ARM CPU**
- **Apple Silicon**
- **Huawei Ascend**
- **AWS Neuron** — Intel XPU

The engine owns scheduling and KV. RuntimeFusion owns FusionCapsules. BackendRegistry binds search and execution to the same backend.

JAX-sklearn (xlearn) — Transparent Acceleration Architecture

User API, compatibility layer, JAX implementations, data policy, and hardware backends

Dashed frame = architecture zone
Solid frame = subsystem
Box = module and responsibility

PROJECT OVERVIEW

JAX-sklearn provides a drop-in scikit-learn API while transparently accelerating supported estimators with JAX.

The architecture has a user API, a compatibility layer, JAX implementations, a data policy, and hardware backends.

USER CODE · sklearn-compatible API

- **import xlearn as sklearn**
  100% call surface
- **Estimators**
  linear_model · cluster
  decomposition · preprocessing
- **Calls**
  fit · predict · transform

COMPATIBILITY · xlearn/__init__.py

- **__getattr__**
  lazy submodule import
- **_auto_jax_accelerate_module**
  replace estimator classes
- **AcceleratorRegistry**
  _accelerator.py
- **create_intelligent_proxy**
  _proxy.py
- **monkey_patch_estimator**
  install proxy class

JAX ACCELERATION · xlearn/_jax

- **EstimatorProxy**
  fit-time dispatch
- **create_accelerated_estimator**
  _accelerator.py
- **_universal_jax.py**
  JIT · vmap
- **original_class**
  sklearn fallback
- **_gp.py**
  Gaussian process
- **_mlp.py**
  neural net
- **_svm.py**
  SVM kernels
- **_tree.py**
  tree models

DATA MANAGEMENT · arrays and policy

- **_data_conversion.py**
  NumPy ↔ JAX
  dtype · shape
- **_config.py**
  enable_jax
  jax_platform
- **jax_auto_threshold**
  complexity >= 1e8
- **detect_hardware**
  install command
  platform probe

HARDWARE · JAX extras

- **CPU**
  jax-cpu · jaxlib
- **CUDA GPU**
  jax-gpu
  jax-cuda11
- **TPU**
  jax-tpu
- **Apple Metal**
  jax-metal
  float32 · experimental

The compatibility layer places EstimatorProxy between the sklearn API and either a JAX implementation or the original estimator.

Secret-Learn — Privacy-Preserving ML Architecture

Applications, sklearn API, FL / SS / SL modes, JAX acceleration, secure devices, and infrastructure

Dashed frame = architecture zone
Solid frame = subsystem
Box = module and responsibility

PROJECT OVERVIEW

Secret-Learn exposes sklearn-compatible machine learning across federated, secret-sharing, and split-learning modes.
The architecture comprises the sklearn API, three privacy modes, JAX acceleration, SecretFlow devices, and infrastructure.

APPLICATION · multi-party use cases

- **Healthcare** — hospital collaboration · patient privacy
- **Finance** — cross-bank analysis · fraud detection
- **IoT / Edge** — edge intelligence · distributed learning
- **Research** — genomics · multi-institution
- **Government** — census analytics · cross-agency data

SKLEARN API · 191 algorithms

- **Linear Models** — 39 · Ridge · Lasso · SGD
- **Ensemble** — 18 · RF · XGBoost
- **Clustering** — 14 · KMeans · DBSCAN
- **Decomposition** — 14 · PCA · NMF · ICA
- **SVM** — 7 · SVC · SVR
- **Neural Networks** — 4 · MLP · BernoulliRBM
- **+ 95 More** — Neighbors · Trees · Naive Bayes

PRIVACY MODES · execution placement

FEDERATED LEARNING

- **Local PYU** — data stays local
- **HEU Aggregate** — secure averaging
- **FLLinearRegression** — federated_learning/
- **Horizontal FL** — 3–5×

SECRET SHARING

- **SPU MPC** — ciphertext compute
- **ABY3 / CHEETAH** — MPC protocols
- **SSPCA** — secret_sharing/
- **Maximum Privacy** — 1–2×

SPLIT LEARNING

- **Model Split** — across parties
- **HEU Activations** — protected tensors
- **SLMLPClassifier** — split_learning/
- **Vertical FL** — 2–4×

JAX ACCELERATION · xlearn bridge

- **JIT / XLA** — traced execution
- **jax.vmap** — batching · autodiff
- **xlearn Backend** — JAX-sklearn bridge
- **Smart Fallback** — original sklearn
- **CPU** — 1.5–2.5×
- **GPU** — 5–8×
- **TPU** — 9–15×
- **M1 / M2** — 2–4×

SECURE DEVICES · SecretFlow

SPU

- **MPC** — ABY3 · CHEETAH
- **Ciphertext** — zero plaintext leak

HEU

- **Schemes** — Paillier · CKKS
- **Aggregation** — gradient protection

TEE

- **Enclaves** — Intel SGX · AMD SEV
- **Occlum** — attestation

PYU

- **Party Local** — data ownership
- **JAX Local** — plaintext compute

INFRASTRUCTURE · runtime stack

- **SecretFlow** — Ray distributed · gRPC · DP-SGD
- **JAX / XLA** — CUDA / cuDNN · TPU runtime
- **sklearn** — NumPy · SciPy

FL, SS, and SL are three placements of the same sklearn API. JAX and SecretFlow devices PYU, HEU, SPU, and TEE form the compute boundary.

YouTube-DNN-RecSys — Candidate Generation Architecture

Data processing, storage, dnn.py model, prediction services, and support utilities

Dashed frame = architecture zone
Solid frame = subsystem
Box = module and responsibility

PROJECT OVERVIEW

YouTube-DNN-RecSys is a TensorFlow and Spark implementation of the candidate-generation stage of YouTube recommendations.
The architecture comprises data processing, storage, the dnn.py model, prediction services, and support utilities.

DATA PROCESSING · offline sample build

Python Pipeline

data2tfrecords.py
random training data

TFRecords
train / evaluation

read_tfrecords.py
dataset reader

Spark Pipeline

MakeDataOne.scala
distributed generation

MakeDataTwo.scala
TFRecords output

HDFS
dataset storage

STORAGE · datasets and checkpoints

data/tfrecords/
train/train.tfrecords
evaluation/evaluation.tfrecords

data/checkpoints/
ckpt/checkpoint
ckpt/eval · modelpath/

MODEL TRAINING · dnn.py

Input
visit_items_index × 5
continuous_features × 16

Item Embedding
500 items × 16

Average Pooling
history vector

DNN Tower
Dense 64 + ReLU
Dropout · Dense 32 + ReLU
user vector 16

Softmax Output
500 classes
cross entropy · Top-K

PREDICTION · serving paths

PredictUserVector.scala
SavedModel
user vector head

ItemEmbeddingPredictor.scala
item embedding table
FeatureBuilder

load_dnn_model.py
SavedModel load
online / batch inference

UTILITIES · training support

tensor_board.py
TensorBoard

feature_column.py
feature spec

self_defined_network_layer.py
custom TF layers

Spark Base
BaseSparkLocal.scala
BaseSparkOnline.scala

Configuration
vars.prod.properties
vars.sit.properties

The model contains input, item embedding, average pooling, the user tower, and softmax. Python and Spark each provide data and prediction components.

Research CLI — Agent System Architecture

CLI, core session and routing, tools and research modules, model providers, and embedded runtimes

Dashed frame = architecture zone
Solid frame = subsystem
Box = module and responsibility

PROJECT OVERVIEW

Research CLI is a terminal-first AI agent for code, tools, MCP services, and academic research workflows.

The architecture separates the CLI, core session and routing, tools and research modules, model providers, and embedded runtimes.

CLI FRONTEND · packages/cli

- **research.tsx** — interactive TUI
- **commands/** — mcp.ts; skills.tsx · hooks.tsx
- **ui/commands/research** — arxivCommand; paper · investigate · submit
- **acpSession.ts** — ACP session; stdio transport

CORE ORCHESTRATION · packages/core

Session

- **researchChat.ts** — researchRequest.ts
- **research-config-manager.ts** — academic settings
- **config / memory** — context window

Routing

- **modelRouterService** — routing/modelRouterService.ts
- **defaultStrategy** — routingStrategy.ts
- **classifierStrategy** — gemma · numerical
- **compositeStrategy** — approval · fallback; override

TOOLS & RESEARCH MODULES

Workspace

- **edit.ts · grep.ts** — filesystem changes
- **shell / ls** — local execution

MCP

- **mcp-client.ts** — server sessions
- **mcp-client-manager.ts** — resource routing

analysis/

- **bibliography-manager** — citations
- **experiment-code-generator** — Python · R · Julia
- **research-data-analyzer** — statistics · ML

bibliography/

- **arxiv-mcp-client** — arXiv search
- **google-scholar-client** — scholar search
- **literature-investigator** — keyword-generator; reference-classifier

submission/

- **latex-manager** — project compile
- **journal-matcher** — venue selection
- **arxiv-latex-extractor** — template extract

MODEL PROVIDERS

- **Gemini / Vertex**
- **OpenAI**
- **Anthropic**
- **Baidu ERNIE**
- **DeepSeek**
- **Qwen**
- **Groq / Mistral** — Cohere · Moonshot

EMBEDDED RUNTIMES

- **packages/sdk** — library embedding
- **packages/a2a-server** — agent executor; HTTP task API
- **vscode-ide-companion** — IDE companion
- **packages/devtools** — development tools

The CLI and ACP form the frontend. Core owns session and model routing. Tools, research modules, providers, and embedded runtimes are separate boundaries.

Questions-Gen — Three-Stage Training Architecture

Training corpus, SFT, GRPO, distillation, validation, and deployment interfaces

Dashed frame = architecture zone
Solid frame = subsystem
Box = module and responsibility

PROJECT OVERVIEW

Questions-Gen is a Qwen3-14B system for generating and evaluating mathematical competition problems.
The architecture contains the training corpus, SFT, GRPO, distillation, validation, and deployment interfaces.

TRAINING DATA · documented mixture

Historical
50%
QuestionDataPreparer.py

Conditional Variants
30%
data_preparer.py

Innovative Types
20%
real_math_data_processor.py

Stage 1 · SFT

Qwen3-14B
base checkpoint

QuestionsGenTrainer.py
src/ + questions_gen/core/trainer.py

stage1-fp-merged
Hugging Face

Stage 2 · GRPO

RewardCalculator.py

Difficulty
0.4 difficulty

Novelty
0.3

Rigor
0.2

Diversity
0.1

Optimization

GRPO Groups
8 problems/group

NoveltyConstraint.py
duplication guard

stage2-merged-16bit
variation generation

Stage 3 · Distill

DeepSeekTeacher.py
DeepSeek-R1

Teacher Signals
difficulty · rigor
innovation · education

final-merged-16bit
Hugging Face

VALIDATION · questions_gen/validation

model_validator.py
single checkpoint

batch_validator.py
algebra · calculus
geometry · number theory

quality_evaluator.py
clarity · difficulty
originality

DEPLOYMENT · interfaces

main_cli.py
validate · batch · quality

hf_utils.py
three published stages

ollama_manager.py
local inference

examples/
quick_demo.py
demo_model_validation.py

SFT, GRPO, and distillation are three model stages. RewardCalculator and NoveltyConstraint belong to GRPO. Validation and deployment are separate interfaces.